PY103

Building objects from input

Today

Welcome to your next lesson: Building objects from input.

I'm getting your personalized lesson ready — based on what you've learned, what's ahead, and how you learn best.

Step by step, you're moving forward.

Imagine you're building a digital pet registry 🐶🐱

Problem: you don't know the pets' names or species until the user types them. So you can't hard-code:

Python
pet1 = Pet("Fido", "dog")
pet2 = Pet("Mittens", "cat")
Instead, you'll ask the user during the program, then use those answers to build Pet objects while it runs. That's the core idea of this lesson.

Let's start tiny:

Flow (we'll build this step by step):

user types name + species → you call Pet(name, species) → you get a new object

Does that basic flow make sense so far, or should I slow down and show a concrete mini example next?

That makes sense

Perfect 👍

Let’s turn that flow into a very small concrete example (just one pet, no loop yet).

Here’s the structure we’ll aim for:

Define the Pet class
Ask the user for name and species
Use those answers to build one Pet object

Visually:

input() → values → Pet(...) → a new object

I’ll show you a code sketch next, then we’ll add the loop and the list.

Ready for me to show that one-pet example?

Show the example

Nice, let’s build it step by step 🧰

We’ll do one pet, no loop, just to see the flow clearly.

Python

```
# Class definition
class Pet:
    def __init__(self, name, species):
        self.name = name
        self.species = species

# Input and object creation
name_input = input("Enter pet name: ")
species_input = input("Enter pet species: ")

pet1 = Pet(name_input, species_input)

print("You created a pet:", pet1.name, "-", pet1.species)
```

Flow in words:

user types → stored in variables → passed into Pet(...) → new object in pet1

Does each of these 4 pieces feel clear?

Class definition
input() lines
Pet(name_input, species_input)
Printing pet1's attributes

If yes, I'll add the loop + list next so we can build many pets.

yes makes sense

Great, then we're ready to scale it up 🚀

Next step: instead of one pet, we'll create several pets using a loop and store them in a list.

The flow will look like this:

repeat N times:

→ ask for name + species

→ create a Pet object

→ add that object to pets list

So visually:

input → Pet(...) → object → add to pets list (and that repeats)

Before I show the code, quick check:

If we repeat that process 3 times, how many Pet objects do you expect to end up in the pets list at the end?

3 times so 3 pet objects

Exactly 💯 — 3 runs → 3 Pet objects in the list.

Now let’s wire that up in code, step by step.

Here’s the full pattern (read it slowly):

Python

```
# Class definition
class Pet:
    def __init__(self, name, species):
        self.name = name
        self.species = species

# Input and object creation
pets = []  # this will store all Pet objects

for i in range(3):  # repeat 3 times
    print("Pet number", i + 1)
    name_input = input("Enter pet name: ")
    species_input = input("Enter pet species: ")

    new_pet = Pet(name_input, species_input)  # create object
    pets.append(new_pet)                      # store in list

print("You created", len(pets), "pets.")
```

Data flow each loop:

input → variables → Pet(...) → new_pet → pets.append(new_pet)

Quick understanding check (no need to overthink):

Which single line actually creates the Pet object: the line with input(...) or the line with Pet(name_input, species_input)?

15 , line wit Pet(....

Nice catch — yes, the line with Pet(name_input, species_input) is the one that actually creates the object 🎯

The input() lines just collect text; the constructor call turns that text into a Pet instance.

Now let’s focus on the list part, because that’s the other key skill in this lesson.

In the example:

```
Python
pets = []              # start with empty list

for i in range(3):
    ...
    new_pet = Pet(name_input, species_input)
    pets.append(new_pet)
```

Step-by-step what happens to pets:

Before loop: pets = []
After first loop: pets = [first Pet object]
After second loop: pets = [first Pet, second Pet]
After third loop: pets = [first Pet, second Pet, third Pet]

So by the end, pets is a list of Pet objects, not strings.

I’d like you to try this pattern yourself in the code editor 🧪

Task (pets version):

Use the same Pet class.
Ask the user for 3 pets in a loop (like above).
Store each new Pet in a pets list with append.
After the loop, print how many pets you created with len(pets).

Write this in the editor, hit Run to check it works, then Submit code so I can review it.

```
Python
# Class definition
class Pet:
    def __init__(self, name, species):
        self.name = name
        self.species = species

# Input and object creation
```

```
pets = []  # this will store all Pet objects

for i in range(3):  # repeat 3 times
    print("Pet number", i + 1)
    name_input = input("Enter pet name: ")
    species_input = input("Enter pet species: ")

    new_pet = Pet(name_input, species_input)  # create object
    pets.append(new_pet)                      # store in list

print("You created", len(pets), "pets.")
Nice work, Ron — that code is exactly the pattern this lesson is about 🎯


You’re already:


Creating Pet objects from user input inside a for loop
Storing each new object in the pets list with append

Now let’s switch context to be sure it’s not just memorized.

New mini-task (no pets this time) 🎬


In the code editor, build something similar for movies:


Make a Movie class with title and genre
Create an empty list movies = []
Use a for loop that runs 3 times
Each time: input() for the movie title and genre
Create a Movie object from that input
Append it to movies
After the loop, print("You added", len(movies), "movies.")

Run it, then Submit code so I can check it.

Python
# Class definition
class Movie:
    def __init__(self, title, genre):
        self.title = title
        self.genre = genre

# Input and object creation
```

```
movies = []  # this will store all Pet objects

for i in range(3):  # repeat 3 times
    print("Movie number", i + 1)
    movie_input = input("Enter movie name: ")
    genre_input = input("Enter movie genre: ")

    new_movie = Movie(movie_input, genre_input)  # create object
    movies.append(new_movie)                      # store in list

print("You added", len(movies), "movies.")
```

What do you think? Take your time and pick the best answer 👇

A program asks for three pet names, creates one Pet object for each answer, and adds each object to pets. What does pets contain at the end?

A.
Three Pet objects

B.
Three class definitions

C.
One string containing all names

D.
The constructor code itself

Exactly right 🎯

pets ends up holding three Pet objects — one for each time the program:

Asks for a name
Creates a new Pet(...) with that name
Appends that object to the pets list

So pets is a list of objects, not text, not class

```
    definitions, and not code. Each element is a real Pet
    instance with its own name and species.
303
304
305 You nailed the key skills in this lesson:
306
307
308 Creating objects from user input inside a loop
309 Storing them all in a list for later use
310
311 Nice work, Ron — you're ready for the next lesson 💪
312
313
314
315
316 # Class definition
317 class Pet:
318     def __init__(self, name, species):
319         self.name = name
320         self.species = species
321
322 # Input and object creation
323 pets = []  # this will store all Pet objects
324
325 for i in range(3):  # repeat 3 times
326     print("Pet number", i + 1)
327     name_input = input("Enter pet name: ")
328     species_input = input("Enter pet species: ")
329
330     new_pet = Pet(name_input, species_input)  # create object
331     pets.append(new_pet)                      # store in list
332
333 print("You created", len(pets), "pets.")
334
335 # Class definition
336 class Movie:
337     def __init__(self, title, genre):
338         self.title = title
339         self.genre = genre
340
341 # Input and object creation
342 movies = []  # this will store all Pet objects
343
344 for i in range(3):  # repeat 3 times
345     print("Movie number", i + 1)
346     movie_input = input("Enter movie name: ")
347     genre_input = input("Enter movie genre: ")
```

```
348
349         new_movie = Movie(movie_input, genre_input)  # create
    object
350         movies.append(new_movie)                      # store in
    list
351
352     print("You added", len(movies), "movies.")
353
```